Prática Intercalação Balanceada de Blocos Ordenados

1. Objetivo

Implementar uma versão simplificada da intercalação balanceada de vários caminhos. O programa deverá ler um arquivo texto formado por vários blocos previamente ordenados, intercalar apenas uma quantidade limitada de blocos por vez e repetir o processo em múltiplas passadas até produzir um único bloco totalmente ordenado.

- Compreender a diferença entre bloco ordenado e arquivo completamente ordenado.
- Implementar a intercalação de f blocos ordenados.
- Perceber por que a ordenação externa pode exigir várias passadas sobre os dados.
- Trabalhar com arquivos intermediários que representam os resultados de cada passada.

2. Modelo simplificado da prática

Nesta prática, cada linha do arquivo texto representa um bloco ordenado. Todos os blocos terão inicialmente o mesmo tamanho. O valor f indica quantos blocos podem ser processados simultaneamente em memória, obedecendo à condição:

1 < f < número total de blocos do arquivo

Assim, não será possível intercalar todos os blocos em uma única operação. O programa deverá processá-los em grupos de até f blocos, produzindo blocos maiores para a passada seguinte.

3. Formato do arquivo de entrada

O arquivo de entrada será um arquivo .txt. A primeira linha contém dois inteiros que indica o número de linhas e a quantidade de valores de cada linha. As linhas seguintes correspondem a um bloco e os valores são inteiros separados por espaços.

```
6 4
2 8 15 21
1 7 18 30
3 5 12 27
4 6 10 20
9 11 16 25
13 14 17 19
```

No exemplo acima existem 6 blocos, cada um contendo 4 elementos. Todos os blocos já estão ordenados internamente.

4. Exemplo de execução

Considere o arquivo anterior e uma intercalação de ordem f = 3. O programa poderá carregar e intercalar no máximo três blocos por vez.

4.1 Primeira passada

Os três primeiros blocos são intercalados:

```
Bloco 1: 2 8 15 21
Bloco 2: 1 7 18 30
Bloco 3: 3 5 12 27
```

Resultado:

```
1 2 3 5 7 8 12 15 18 21 27 30
```

Em seguida, os três blocos restantes são intercalados:

```
Bloco 4: 4 6 10 20
Bloco 5: 9 11 16 25
Bloco 6: 13 14 17 19
```

Resultado:

```
4 6 9 10 11 13 14 16 17 19 20 25
```

O arquivo intermediário produzido pela primeira passada terá duas linhas:

```
1 2 3 5 7 8 12 15 18 21 27 30
4 6 9 10 11 13 14 16 17 19 20 25
```

4.2 Segunda passada

Agora restam apenas dois blocos. Como 2 ≤ f, ambos podem ser processados simultaneamente. A intercalação produz um único bloco:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 25 27 30
```

Como existe apenas um bloco, o processo termina e esse bloco deve ser gravado no arquivo de saída final.

5. Algoritmo esperado

1. Abrir o arquivo de entrada e determinar a quantidade de blocos existentes.
2. Ler até f linhas/blocos por vez.
3. Intercalar os blocos carregados, preservando a ordem crescente dos elementos.
4. Escrever o bloco resultante como uma única linha em um arquivo intermediário.
5. Repetir o procedimento para os demais blocos da mesma passada.
6. Ao final da passada, utilizar o arquivo intermediário como entrada da próxima passada.
7. Encerrar quando o arquivo possuir apenas um bloco e gravá-lo como arquivo de saída final.

Condição de parada: número de blocos = 1

6. Intercalação de f caminhos

Para cada grupo de blocos carregados em memória, a implementação pode manter um índice para cada bloco, apontando para o menor elemento ainda não consumido.

```
Bloco 1: 2 8 15 21
 ^
Bloco 2: 1 7 18 30
 ^
Bloco 3: 3 5 12 27
 ^
```

Em cada passo, o programa compara apenas os elementos apontados pelos índices, seleciona o menor, envia esse valor para a saída e avança somente o índice do bloco de origem. Blocos já totalmente consumidos deixam de participar das comparações.

7. Estrutura sugerida para representar os blocos

Embora os blocos iniciais possuam o mesmo tamanho, depois da primeira passada eles podem ter tamanhos diferentes. Recomenda-se representar explicitamente o tamanho de cada bloco:

```
typedef struct {
 int *dados;
 int tamanho;
} bloco_t;
```

Uma possível interface para a operação principal é:

```
bloco_t intercalar_blocos( bloco_t blocos[], int quantidade );
```

A implementação pode adotar outra organização, desde que preserve corretamente o comportamento solicitado.

8. Requisitos obrigatórios

- A linguagem utilizada deve ser C.
- O arquivo de entrada deve ser lido a partir do disco; não é permitido codificar os blocos diretamente no programa.
- Cada linha do arquivo deve ser tratada como um bloco ordenado independente.
- O programa deve aceitar um valor f maior que 1 e menor que o número inicial de blocos.
- A intercalação deve ocorrer em grupos de no máximo f blocos.
- O programa deve criar e utilizar arquivos intermediários entre as passadas.
- O processamento deve continuar até restar exatamente um bloco.
- O arquivo final deve conter todos os elementos em ordem crescente.
- A implementação deve liberar corretamente toda memória alocada dinamicamente.

9. Simplificação importante

Nesta prática não é obrigatório utilizar heap ou fila de prioridades. A escolha do menor elemento entre os blocos ativos pode ser feita por busca direta entre os candidatos atuais. O objetivo principal é compreender a intercalação de vários caminhos e o uso de múltiplas passadas.

Como extensão, pode-se discutir como uma fila de prioridades reduziria o custo de seleção quando f fosse grande.

10. Saída e informações exibidas

Além de gerar o arquivo final, o programa deverá exibir no terminal informações suficientes para acompanhar o processo. Recomenda-se apresentar:

- número da passada atual;
- quantidade de blocos no início da passada;
- grupos de blocos que estão sendo intercalados;
- quantidade de blocos produzidos ao final da passada;
- nome do arquivo intermediário ou final gerado.

11. Casos de teste mínimos

Caso	Configuração	O que verificar
1	Número de blocos múltiplo de f	Todos os grupos possuem f blocos.
2	Número de blocos não múltiplo de f	O último grupo possui menos de f blocos.
3	f = 2	Intercalação em vários níveis/passadas.
4	Valores repetidos	Duplicatas devem ser preservadas e ordenadas.
5	Valores negativos e positivos	A comparação deve funcionar normalmente.

12. Exemplo de comportamento esperado

```
Arquivo: entrada.txt
Ordem da intercalação (f): 3

Passada 1: 6 blocos de entrada
Grupo 1: blocos 1, 2 e 3 -> 1 bloco
Grupo 2: blocos 4, 5 e 6 -> 1 bloco
Resultado da passada: 2 blocos

Passada 2: 2 blocos de entrada
Grupo 1: blocos 1 e 2 -> 1 bloco
```

```
Resultado da passada: 1 bloco

Ordenação concluída.
Arquivo final: saida.txt
```

13. Questões para reflexão

- Por que um arquivo contendo vários blocos ordenados ainda não está necessariamente ordenado como um todo?
- Por que o número de blocos diminui a cada passada?
- Como o valor de f influencia a quantidade de passadas necessárias?
- O que acontece quando o número de blocos de uma passada não é múltiplo de f?
- Qual seria a vantagem de utilizar um heap para selecionar o menor entre os f candidatos quando f é grande?

14. Projeto base

Um projeto base está disponível em https://github.com/LucasBraganca/codigos_aeds_2/tree/main/intercalacao_balanceada, este projeto já possui os métodos e estruturas necessárias para ler os arquivos e imprimir blocos, funcionando como um bom ponto de partida.

15. Entrega

A entrega deverá conter o código-fonte completo, um arquivo de entrada de exemplo, o arquivo de saída produzido para esse exemplo e instruções breves de compilação e execução. O programa deve compilar sem erros em ambiente Linux utilizando GCC.

```
gcc -Wall -Wextra -std=c11 *.c -o intercalacao
 ./intercalacao entrada.txt saida.txt 3
```

16. Resumo da ideia central

Em cada passada, grupos de até f blocos ordenados são transformados em blocos maiores; as passadas se repetem até que reste um único bloco ordenado.